

Creative ways to automate PWA backups to local filesystems

No web API today lets a PWA silently write to the user-visible filesystem without a user gesture—that's the core constraint driving every creative solution here. Each approach below works around this sandbox limitation through a different architectural bridge: a local server, a cloud API, a browser extension, or a native wrapper. The right choice depends on your user base, how much installation friction you can tolerate, and whether "local filesystem" is a hard requirement or "durable, user-owned backup" is the real goal. All of these approaches have been seriously proposed, discussed, or shipped in real projects.

The local companion server is the most flexible bridge

A small HTTP server binary running on `localhost` accepts POST requests from the PWA via standard `fetch()`. Since browsers treat `localhost` as a secure context, no HTTPS is needed.

Mozilla The PWA serializes its data (JSON, SQLite export, ZIP) and sends it to something like `http://127.0.0.1:8080/backup`, where the companion writes it to disk.

Implementation is surprisingly minimal. A Python script extending `http.server.BaseHTTPRequestHandler` with a `do_POST` method can do this in roughly 20 lines. Go and Rust compile to zero-dependency static binaries that users can download and double-click. Node.js with Express works too. The server binds exclusively to `127.0.0.1` to prevent external access, and CORS headers whitelist only the PWA's origin.

Tauri's sidecar feature provides a production-ready framework for this pattern. A compiled binary (Go, Rust, Python via PyInstaller) runs alongside the app and communicates through localhost HTTP, TCP sockets, stdin/stdout, or gRPC. Evil Martians documented using a long-lived Tauri sidecar with TCP socket communication for filesystem monitoring, demonstrating this works well in practice.

The tradeoffs are straightforward: this works in **every browser** since it only uses `fetch()`, but it requires users to install and run a native binary—significant friction. You can mitigate this with auto-start on login (systemd, Windows Startup, macOS LaunchAgent) and a one-click installer. Security-wise, any local process on the machine could theoretically POST to your server, so adding a shared secret or auth token is wise. The developer tools ecosystem (VS Code's local server, browser-sync, webpack-dev-server) proves this pattern works reliably at scale.

WebDAV adds protocol-level file management for free

WebDAV extends HTTP with file management verbs (PUT, MKCOL, PROPFIND, LOCK), turning the companion server pattern into something more powerful. The PWA uses a JavaScript WebDAV client to create directories, write files, check metadata, and manage backups—all through standardized HTTP methods.

The [webdav](#) [npm package](#) ([perry-mitchell/webdav-client](#)) is the most mature browser-compatible client library, [npm](#) providing [putFileContents\(\)](#), [getDirectoryContents\(\)](#), [createDirectory\(\)](#), and more via fetch/XHR. On the server side, **rclone** provides a single-binary, cross-platform WebDAV server out of the box: [rclone serve webdav ~/pwa-backups/ --addr 127.0.0.1:8080 --allow-origin https://my-pwa.com](#). It supports Basic auth, TLS, and CORS configuration.

The key advantage over a plain HTTP server is that WebDAV is a standardized protocol understood by Windows Explorer, macOS Finder, and Linux file managers. Users can mount the backup directory as a network drive alongside accessing it from the PWA. Rclone can also front cloud storage backends (S3, Google Drive), meaning the same WebDAV endpoint could write to local disk *and* cloud storage simultaneously. The building blocks are mature—Nextcloud and ownCloud rely on WebDAV extensively—though no one has packaged a dedicated "PWA backup via local WebDAV" project yet.

Cloud storage APIs bypass the filesystem entirely

If the real goal is "durable, user-owned backup" rather than literally writing to the local filesystem, **cloud storage APIs offer the broadest browser compatibility and the smoothest automation story**. Google Drive, Dropbox, and OneDrive all expose REST APIs that a PWA can call directly from the browser after OAuth authentication.

The Google Drive approach is well-proven. After registering in Google Cloud Console and using the Google Identity Services library for browser-side OAuth, the PWA uploads backup files via multipart POST to [googleapis.com/upload/drive/v3/files](#). The **voice-to-drive** project ([carlorbiz/voice-to-drive](#)) is a working PWA that records voice notes with automatic Google Drive sync, [github](#) using a dedicated [drive.js](#) module alongside IndexedDB for local storage. [GitHub](#) Dropbox provides an official JavaScript SDK supporting browser-based auth and upload. [Reintech](#) Microsoft's Graph API handles OneDrive via MSAL.js.

A particularly creative hybrid: **use the File System Access API to save to a cloud-synced local folder**. The user picks their Google Drive or OneDrive sync folder as the save target, and writes to local files that auto-sync to the cloud. [Medium](#) This gives you local filesystem backup and cloud backup simultaneously, with no server-side API registration needed.

The open-source alternative is the remoteStorage protocol, an IETF Internet-Draft specifically designed for "unhosted" web apps. [Unhosted](#) It combines WebFinger discovery, OAuth, and CORS-enabled REST storage. [Remotestorage](#) The [remotestorage.js](#) client library provides offline-first sync with data modules, and can fall back to Dropbox or Google Drive as backends. [Unhosted](#) The tradeoff is that users need a remoteStorage provider, which remains niche.

Cloud API costs are reasonable: Google and Microsoft offer **free tiers up to roughly 50,000 users**. [Medium](#) [medium](#) The main friction points are OAuth consent screens (users must grant access to their cloud account) and the requirement for internet connectivity during backup.

Browser extensions create a privileged bridge to the filesystem

Extensions operate in a privileged context with APIs unavailable to regular web pages.

[Chrome Developers](#) The most powerful pattern is a **three-layer architecture**: PWA → Extension → Native Messaging Host → Filesystem.

The PWA communicates with a companion extension via

`chrome.runtime.sendMessage(extensionId, data)`, enabled by the extension declaring

`"externally_connectable"` for the PWA's origin. The extension then uses **Chrome's Native**

Messaging API to communicate with a registered native binary via stdin/stdout JSON

messages. [MDN Web Docs +2](#) The native app (a simple Python or Node.js script) handles actual

filesystem writes. [MDN Web Docs](#) The [simov/native-messaging](#) npm package provides a Node.js

implementation of the native messaging protocol. [GitHub](#)

A simpler extension-only approach exists: the **Site Storage Backup** Chrome extension backs up a website's localStorage and cookies into an encrypted `.enc` file, [Chrome Web Store](#) with

IndexedDB support planned. [WebExtension.ORG](#) The **OPFS Explorer** DevTools extension

demonstrates that extensions can reach into OPFS to inspect, download, and delete files.

[GitHub +2](#)

A creative idea from Scott Kuhl's "PWA Data Trap" article: **use browser bookmarks as a sync mechanism**. An extension stores structured data in bookmarks using the Bookmarks API,

leveraging the browser's built-in bookmark sync across devices. [Medium](#) [medium](#) The Yawas

highlighter extension uses this pattern for cross-device data persistence. [Medium](#)

The friction is substantial: users must install both an extension and (for native messaging) a native binary. Extensions are browser-specific, with Chrome Manifest V3, Firefox WebExtensions, and Safari having different APIs. But once installed, this approach provides **fully automated, background filesystem writes** with no per-operation user gesture required.

Tauri wrappers outperform Electron for adding native capabilities

When PWA-native APIs are insufficient, wrapping the web app in a native shell is the "escape

hatch." [Medium](#) **Tauri has emerged as the clear winner over Electron** for this use case. It

uses the OS's native webview instead of bundling Chromium, resulting in app sizes of **~30MB**

versus Electron's ~200MB. [Philrich](#) Its Rust backend provides memory safety, and its

capability-based permission system allows fine-grained filesystem access control.

Tauri's [@tauri-apps/plugin-fs](#) provides `readTextFile()`, `writeTextFile()`, `readDir()`, and other

filesystem operations with built-in path traversal prevention. [Tauri](#) Each window can have

different permissions defined in capability files, and capabilities can be platform-specific.

[Tauri](#) The Betaflight Configurator project has an active proposal (GitHub Issue #4499) to

transition from a browser PWA to a Tauri 2.0 desktop app specifically for USB serial,

filesystem, and mDNS capabilities. [GitHub](#)

Capacitor (Ionic) deserves mention for mobile targets. [Medium](#) It wraps web apps in native

iOS/Android containers, exposing native filesystem APIs through TypeScript. [Medium](#) [Ctologic](#)

The critical limitation: when running as a pure PWA (not wrapped in a native container),

Capacitor's filesystem plugin falls back to IndexedDB—not true filesystem access.

[Ionic Framework](#) [Ionic Framework](#)

A smart architectural pattern from the Aurelia framework docs enables a **single codebase to adapt behavior across runtimes**:

- Detect `window.__TAURI__` for Tauri environment
- Detect `window.electronAPI` for Electron
- Check `window.matchMedia('(display-mode: standalone)')` for installed PWA
- Fall back to web export for plain browser (Aurelia)

This lets you ship a PWA that automatically upgrades its backup capabilities when running inside a native wrapper.

Service Workers and Background Sync offer partial automation

Service Workers can intercept data-mutation requests, (web.dev) clone them, and accumulate backup payloads in IndexedDB or OPFS. (SimiCart) (web.dev) Combined with the **Background Sync API**, a service worker can POST accumulated data to a localhost companion server even after the PWA window closes. The (sync) event fires when the browser detects network connectivity, (Mozilla) (Monterail) and Google's Workbox library provides (workbox-background-sync) for this exact pattern.

Periodic Background Sync goes further, allowing registration of recurring tasks ((registration.periodicSync.register('backup', { minInterval: 86400000 }))). But the limitations are severe: it's **Chrome-only**, requires the PWA to be installed, and the browser throttles frequency based on a "site engagement score." (Felix Gerschau) Low-engagement PWAs may never get periodic sync events. (Felix Gerschau) Apple does not support Background Sync at all on iOS/Safari. (Apple Developer)

The fundamental constraint: **service workers cannot access the File System Access API** (it requires a user gesture and a visible window). They can write to OPFS or IndexedDB, but not to user-visible filesystem locations. (Medium) The practical combination is to use service workers for data accumulation and preparation, then trigger the actual filesystem export when the user next opens the app.

Why OPFS alone won't solve the backup problem

OPFS (Origin Private File System) provides high-performance, no-permission-prompt file storage—** (web.dev) (MDN Web Docs) 3-4x faster than IndexedDB**—and supports SQLite WASM backends. (RxDB) (RxDB) Notion, Kiwix, and DuckDB all use it in production. (GitHub) (Web.dev) But OPFS files are explicitly **not intended to be user-accessible**. (MDN Web Docs) (Chrome Developers)

Pointing Syncthing or another sync tool at Chrome's internal File System directory is unreliable because Chromium stores OPFS in an opaque database format, not as individual files. (web.dev) Firefox uses SQLite for metadata with a slightly more accessible structure under (~/.mozilla/firefox/<profile>/storage/default/<SITE>/fs/), (A Weird Imagination) but the internal format can change between browser versions and files may be locked while the browser runs.

The viable hybrid: use OPFS for fast working storage, then periodically export snapshots (JSON, SQLite DB, ZIP) to a user-selected folder via any of the other approaches described here. The export target can be a Syncthing-watched folder, providing automatic replication without the fragility of directly syncing browser internals. [Medium](#) Use `navigator.storage.persist()` to prevent the browser from evicting OPFS data under storage pressure. [What PWA Can Do +2](#)

Choosing the right approach for your situation

The decision tree comes down to three factors: **how much installation friction your users will tolerate, whether you need true local filesystem writes or just durable backups, and which browsers you must support.**

- **Zero installation friction, cross-browser:** Cloud storage APIs (Google Drive/Dropbox) are your best bet. OAuth is the only user friction, and backup is fully automated once authenticated.
- **Local filesystem required, technical users:** A companion HTTP server or WebDAV via rclone works in every browser and gives full filesystem control. Package it with auto-start scripts.
- **Local filesystem required, non-technical users:** A Tauri wrapper adds native capabilities at ~30MB cost. [Philrich](#) Ship both a PWA and a Tauri desktop app from the same codebase.
- **Maximum automation, Chromium-only acceptable:** Combine OPFS for working storage with Periodic Background Sync for data preparation and a companion extension with Native Messaging for filesystem writes.
- **Mobile-first:** Cloud APIs are the only practical automated option. iOS blocks Background Sync, [Apple Developer](#) limits OPFS, and does not support File System Access API.

No single solution fully bridges the web sandbox to the local filesystem without tradeoffs. The most robust production strategy combines multiple approaches: [Medium](#) **OPFS or SQLite WASM for high-performance persistence, cloud API sync for automated durable backup, and a local companion server or Tauri wrapper as an optional upgrade** for users who want true local filesystem control. This layered approach degrades gracefully across browsers and platforms while giving power users the local backup capabilities they expect.